

使用 ADK 建立多代理系統、部署至 Agent Engine，並開始使用 A2A 通訊協定

來源：<https://codelabs.developers.google.com/codelabs/create-multi-agents-adk-a2a?hl=zh-tw>

步驟數：12

在本實驗室中，您將使用 ADK 和 A2A 建立多代理系統，並部署至 Google Cloud (Vertex AI Agent Engine)。

目錄

1. 本實驗室的目標
2. 專案設定
3. 啟用 API
4. Agent Development Kit 簡介
5. Vertex AI Agent Engine 簡介
6. A2A 簡介
7. 代理程式架構
8. 安裝 ADK 並設定環境
9. 部署至 Agent Engine
10. 建立 A2A 代理程式
11. 清除所用資源
12. 結論

步驟 1 · 約 0 分鐘

1. 本實驗室的目標

在本實作實驗室中，您將使用 [ADK \(Agent Development Kit\)](#) 建構多代理應用程式，根據提示生成圖片，並根據提示評估圖片。如果生成的圖片不符合提示中的規定，智慧助理會持續生成圖片，直到生成符合規定的圖片為止。本實作實驗室中的每個代理程式都有單一用途，這些代理程式會互相合作，以達成整體目標。您將瞭解如何在本機測試應用程式，並在 [Vertex AI Agent Engine](#) 中部署。

課程內容

- 瞭解 [ADK \(Agent Development Kit\)](#) 的基本概念，以及如何建立多代理系統。
 - 瞭解如何輕鬆在 [Vertex AI Agent Engine](#) 中部署及使用代理。
 - 瞭解 [A2A 通訊協定](#) 的基本概念
 - 瞭解如何搭配使用 [A2A 通訊協定](#) 和 [ADK \(Agent Development Kit\)](#)，建立開放代理。
-

2. 專案設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

- 如果沒有可用的[專案](#)，請在 [GCP 主控台](#) 中建立新專案。
- 在本實驗室中，我們將使用 [GCP Cloud Shell](#) 執行工作。開啟 Cloud Shell，並使用 Cloud Shell 設定專案。
- 按一下[這裡](#)開啟 [GCP Cloud Shell](#)。如果看到「授權 Shell」彈出式視窗，請點選授權 Cloud Shell 編輯器。
- 您可以在 Cloud Shell 終端機中執行下列指令，檢查專案是否已通過驗證。

```
gcloud auth list
```

- 在 Cloud Shell 中執行下列指令，確認專案

```
gcloud config list project
```

- 複製專案 ID，然後使用下列指令設定

〇〇〇〇

```
gcloud config set project <YOUR_PROJECT_ID>
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

〇〇〇〇

```
gcloud projects list
```

步驟 3 · 約 0 分鐘

3. 啟用 API

我們需要啟用一些 API 服務，才能執行本實驗室。在 Cloud Shell 中執行下列指令。

```
gcloud services enable aiplatform.googleapis.com
gcloud services enable cloudresourcemanager.googleapis.com
```

API 簡介

- Vertex AI API (`aiplatform.googleapis.com`) 可存取 Vertex AI 平台，讓應用程式與 Gemini 模型互動，生成文字、進行聊天工作階段及呼叫函式。
 - Cloud Resource Manager API (`cloudresourcemanager.googleapis.com`) 可讓您以程式輔助方式管理 Google Cloud 專案的中繼資料，例如專案 ID 和名稱。其他工具和 SDK 通常需要這些資料，才能驗證專案身分和權限。
-

4. Agent Development Kit 簡介

Agent Development Kit 為需要建構代理應用程式的開發人員帶來多項關鍵優勢：

- 1. **多代理系統**：可在階層結構中組合多個專用代理，建構出可擴充的模組化應用程式，藉以執行複雜的協調和委派作業。
- 2. **豐富的工具生態系統**：為代理提供多元功能，像是使用預先建構的工具 (搜尋、程式碼執行等)、建立自訂函式、整合第三方代理框架的工具 (LangChain、CrewAI)，甚至還可以使用其他代理做為工具。
- 3. **彈性的自動化調度管理功能**：使用工作流程代理 (SequentialAgent 、 ParallelAgent 和 LoopAgent) 定義可預測的管道工作流程，或透過使用 LLM 的動態轉送功能 (LlmAgent 轉移)，創造靈活應變的代理。
- 4. **整合式開發人員體驗**：使用功能強大的 CLI 和互動式開發 UI，在本機開發、測試及偵錯，逐步檢查事件、狀態和代理執行作業。
- 5. **內建評估功能**：與預先定義的測試案例比較，評估最終回覆品質和逐步執行軌跡，有系統地判斷代理效能。
- 6. **可隨時部署**：將代理容器化並部署至任何位置，無論是在本機執行、透過 Vertex AI Agent Engine 擴充，或使用 Cloud Run/Docker 整合至自訂基礎架構都沒問題。

雖然其他生成式 AI SDK 或代理框架同樣能查詢模型，甚至為模型提供工具，但您必須投入大量心力，在多個模型間靈活調度。

相較之下，Agent Development Kit 比上述工具更加高階，您可以輕鬆相互連結多個代理，建立複雜但方便維護的工作流程。

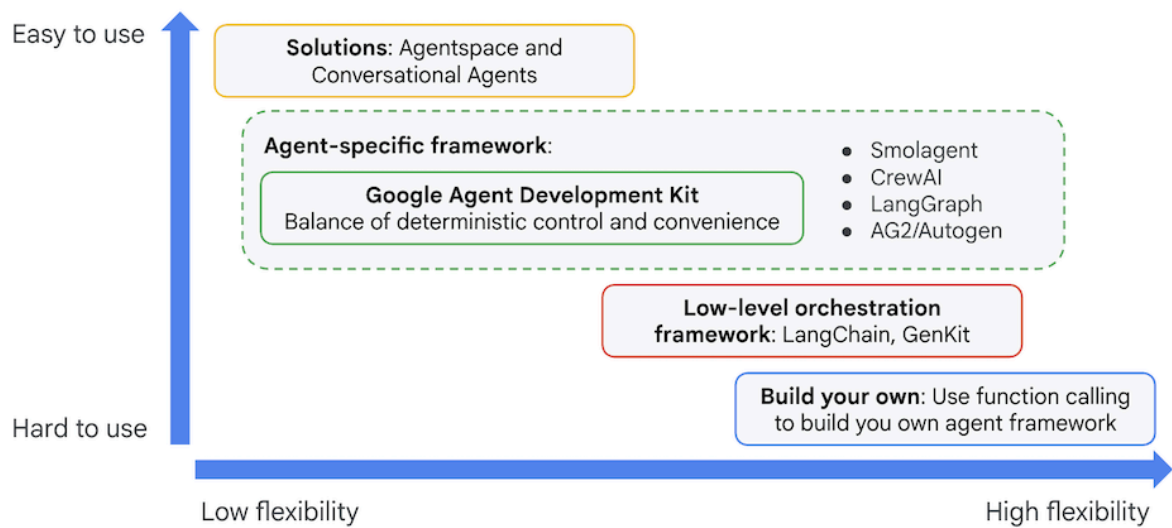


圖 1：ADK (Agent Development Kit) 的定位

5. Vertex AI Agent Engine 簡介

[Vertex AI Agent Engine](#) 是全代管服務，可在 Google Cloud 中部署代理程式。開發人員可透過 [Vertex AI Agent Engine](#)，在 [Vertex AI](#) 上開發、自訂、部署、提供及管理 OSS AI 代理([ADK \(Agent Development Kit\)](#)、LangChain、LangGraph、CrewAI、AutoGen 等！)。

Agent Engine 也提供服務來處理使用者資料，也就是所謂的代理程式記憶體。目前提供兩種記憶體服務。

- **短期記憶**：透過 [Agent Engine 工作階段](#)，您可以在單一工作階段中儲存、管理及擷取進行中的對話記錄 (狀態)，做為短期記憶。
- **長期記憶**：透過 [Agent Engine Memory Bank](#)，儲存、轉換及擷取記憶 (狀態)，特別是跨多個工作階段的長期記憶。

您也可以 Cloud Run 或 GKE 等其他 Google Cloud 服務中部署 Agent，但建議針對下列用途使用 [Vertex AI Agent Engine](#)。

- **具狀態的代管執行階段**：如果需要具狀態的全代管執行階段來部署代理程式，建議使用 [Vertex AI Agent Engine](#)，因為這項服務會將常見工作 (例如工作階段管理、AI 代理程式的持續性) 抽象化。
- **程式碼執行**：如果 Agent 需要執行在使用者工作階段期間動態生成的程式碼，Agent Engine 會提供安全的 Sandbox，供您執行程式碼。
- **彈性的長期記憶**：如果需要為代理提供彈性的長期記憶，可搭配 [Vertex AI Agent Engine](#) 使用的 Vertex AI Memory Bank，就能彈性記憶使用者資訊，並在不同工作階段中使用。

您也可以將 [Vertex AI Agent Engine](#) 與 [Cloud Run](#) 等其他執行階段結合，建立彈性的應用程式架構。以下是使用各種服務建構代理程式的參考架構範例。

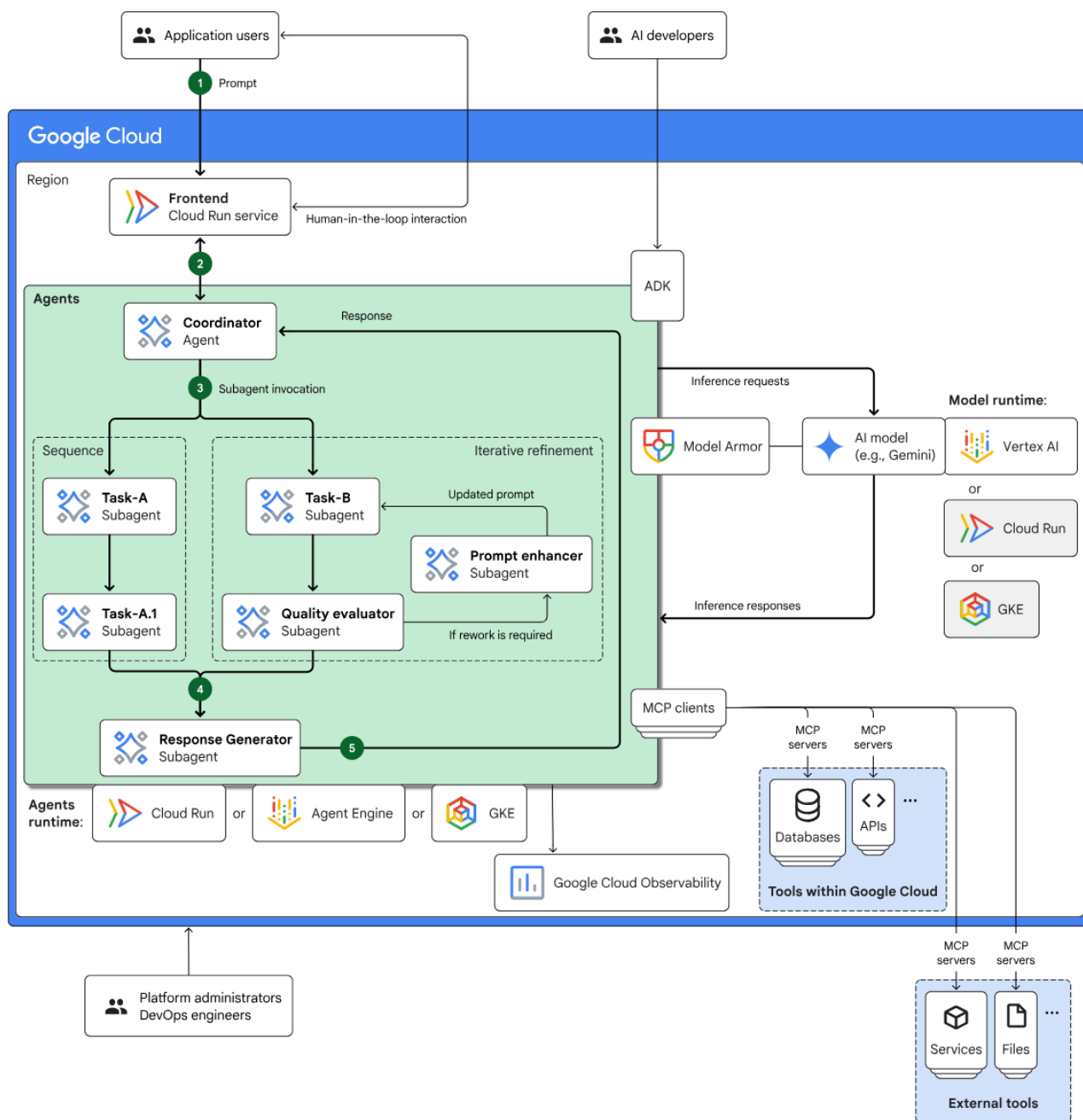


圖 2：使用多項服務建構 Agent 的參考架構範例。

6. A2A 簡介

Agent2Agent (A2A) 通訊協定是一項開放標準，旨在讓不同架構、供應商和網域的自主 AI 代理，能夠流暢且安全地通訊及協作。

- 普遍互通性：**A2A 可讓代理協同運作，不受底層技術限制，打造真正的多代理生態系統。也就是說，不同公司在不同平台上建構的代理程式可以通訊及協調。
 - 能力探索：**代理可以使用「代理資訊卡」(JSON 文件) 宣傳能力，說明身分、支援的 A2A 功能、技能和驗證需求。這樣一來，其他代理程式就能找出並選取最適合特定工作的代理程式。
 - 預設採用安全設定：**安全性是核心原則。A2A 採用企業級驗證和授權機制，並使用 HTTPS/TLS、JWT、OIDC 和 API 金鑰等標準，確保互動安全無虞，並保護機密資料。
 - 不限模式：**通訊協定支援各種通訊模式，包括文字、音訊和視訊串流，以及互動式表單和嵌入式 iframe。代理程式可彈性地以最適合工作和使用者的格式交換資訊。
 - 結構化工作管理：**A2A 會明確定義工作委派、監控和完成的通訊協定。這項功能支援將相關工作分組，並使用不重複的工作 ID，在不同代理之間管理工作。工作可以經歷定義的生命週期 (例如已提交、處理中、已完成)。
 - 不透明的執行作業：**這項重要功能是指代理不必向其他代理揭露內部推論程序、記憶體或特定工具。這些服務只會公開可呼叫的服務，有助於提升模組化程度和隱私權。
 - 以現有標準為基礎：**A2A 採用 HTTP、伺服器傳送事件 (SSE) 即時串流和 JSON-RPC 結構化資料交換等成熟的網路技術，因此更容易與現有 IT 基礎架構整合。
 - 非同步通訊：**通訊協定的設計主要考量非同步通訊，可促進彈性工作進度，即使連線未持續維護，也能推送更新通知。
-

7. 代理程式架構

在本實驗室中，您將建立多代理程式應用程式，根據規格生成圖片並評估圖片，然後向您呈現。

系統架構包含一個名為 **image_scoring** 的主要代理程式，負責協調整個程序。這個主要代理有一個名為 **image_generation_scoring_agent** 的子代理，而這個子代理本身也有自己的子代理，負責執行更具體的任務。這會建立階層式關係，主代理會將工作委派給子代理。

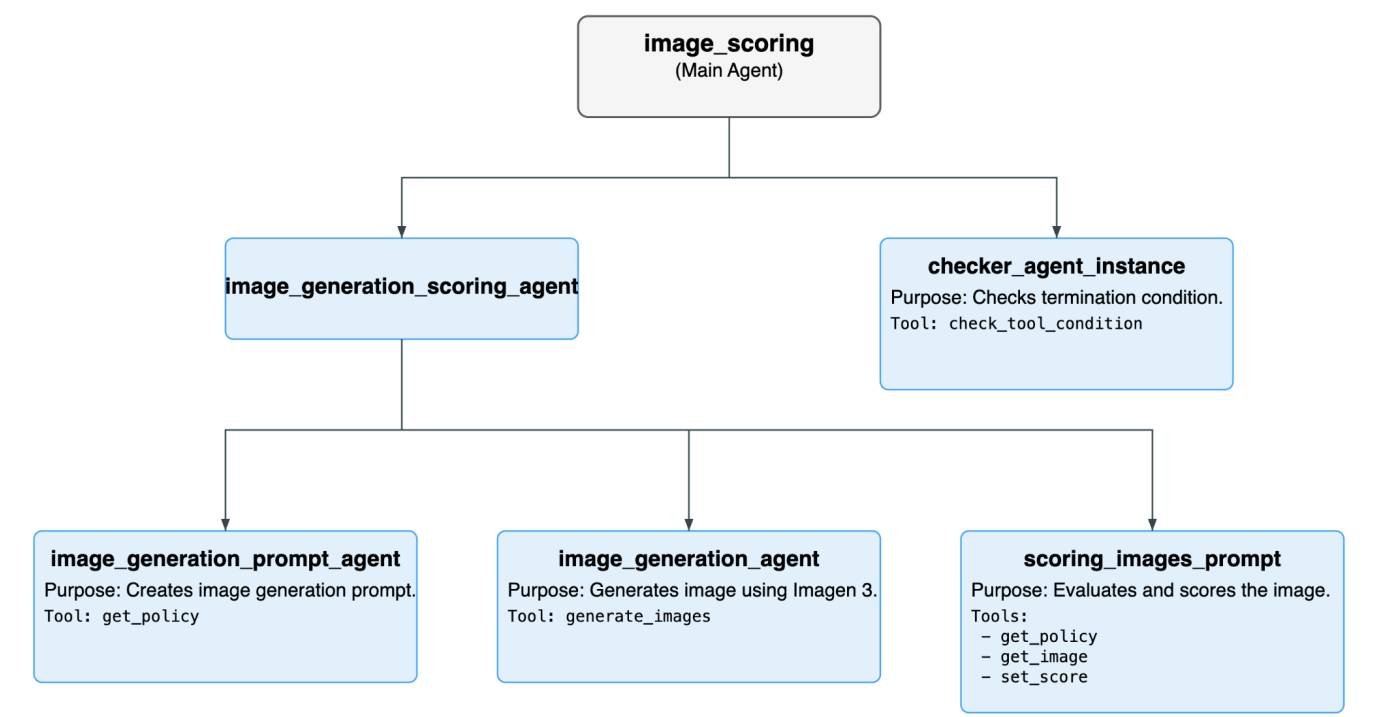


圖 3：整體代理程式流程。

所有代理程式的清單

Agent	Purpose	子代理程式
image_scoring (Main Agent)	這是管理整體工作流程的根代理程式。系統會以迴圈方式反覆執行 image_generation_scoring_agent 和 checker_agent ，直到符合終止條件為止。	image_generation_scoring_agent checker_agent_instance
image_generation_scoring_agent (image_scoring 的子代理)	這個代理負責生成及評估圖片的核心邏輯。為此，這項代理程式會執行一連串的子代理。	image_generation_prompt_agent image_generation_agent scoring_images_prompt
checker_agent_instance (image_scoring 的子代理程式)	這個代理程式會檢查是否應繼續或終止圖片評分程序。這項工具會使用 check_tool_condition 工具評估終止條件。	-

checker_agent_instance (image_scoring 的子代理程式)	這個代理程式擅長建立圖像生成提示，這項工具會接收輸入文字，並生成適合圖片生成模型的詳細提示。	-
image_generation_prompt_agent (image_generation_scoring_agent 的子代理)	這個代理程式擅長建立圖像生成提示，這項工具會根據輸入文字生成詳細提示詞，供圖像生成模型使用。	-
scoring_images_prompt (image_generation_scoring_agent 的子代理程式)：	這個代理程式擅長根據各種條件評估圖片並給予分數。並為生成的圖片評分。	-

使用的所有工具清單

工具	說明	使用者代理程式
check_tool_condition	這項工具會檢查是否符合迴圈終止條件，或是否已達到疊代次數上限。如果其中任一項為 true，迴圈就會停止。	checker_agent_instance
generate_images	這項工具會使用 Imagen 3 模型生成圖片。也可以將生成的圖片儲存至 Google Cloud Storage bucket。	image_generation_agent
get_policy	這項工具會從 JSON 檔案擷取政策。這項政策會由 image_generation_prompt_agent 用來建立圖片生成提示，並由 scoring_images_prompt 用來為圖片評分。	image_generation_agent
get_image	這項工具會載入生成的圖像構件，以便評分。	scoring_images_prompt
set_score	這項工具會在工作階段狀態中設定生成圖片的總分。	scoring_images_prompt

步驟 8 · 約 0 分鐘

8. 安裝 ADK 並設定環境

在本實作實驗中，我們將使用 Cloud Shell 執行工作。

準備 Cloud Shell 編輯器分頁

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 按一下「繼續」。
3. 系統提示您授權 Cloud Shell 時，點選「授權」。
4. 操作本實驗室其餘步驟時，您可以全程將這個視窗當成 IDE 使用，搭配 Cloud Shell 編輯器和 Cloud Shell 終端機作業。
5. 在 Cloud Shell 編輯器中，依序點選「Terminal」(終端機) > 「New Terminal」(新增終端機)，開啟新的終端機。下列所有指令都會在這個終端機上執行。

下載並安裝本實驗室所需的 ADK 和程式碼範例

1. 執行下列指令，從 GitHub 複製所需來源，並安裝必要程式庫。在 Cloud Shell 編輯器中開啟的終端機中執行指令。

```
#create the project directory
mkdir ~/imagescoring
cd ~/imagescoring
#clone the code in the local directory
git clone https://github.com/haren-bh/multiagenthandson.git
```

2. 我們會使用 uv 建立 Python 環境 (在 Cloud Shell 編輯器終端機中執行)：

```
#Install uv if you do not have installed yet
pip install uv

#Create the virtual environment
uv venv .adkvenv

source .adkvenv/bin/activate

#go to the project directory
cd ~/imagescoring/multiagenthandson

#install dependencies
uv pip install -r pyproject.toml
```

3. 如果沒有雲端儲存空間值區，請在 [Google Cloud Storage](#) 中建立新的值區。您也可以使用 gsutil 指令建立 bucket。授予 Agent Engine Google Cloud Storage 存取權 (在 Cloud Shell 編輯器終端機中執行)。

```
# First, make sure your PROJECT_ID variable is set
PROJECT_ID=$(gcloud config get-value project)

# Now, create the bucket with a unique name
# We'll use the project ID to help ensure uniqueness
gsutil mb gs://${PROJECT_ID}-imagescoring-bucket

#Now lets give Agent Engine the permission to access Cloud Storage
# 1. Get the current Project ID (text) and Project Number (numeric)
PROJECT_ID=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")

# 2. Construct the Reasoning Engine Service Account email
SA_EMAIL="service-${PROJECT_NUMBER}@gcp-sa-aiplatform-re.iam.gserviceaccount.com"
# 3. Create Agent Engine Service account if not already created
gcloud beta services identity create --service=aiplatform.googleapis.com --
project=${PROJECT_NUMBER}

# 3. Grant GCS Access
gcloud projects add-iam-policy-binding $PROJECT_ID --member="serviceAccount:$SA_EMAIL" --
role="roles/storage.objectUser" --condition=None
```

4. 在編輯器中，依序前往「View」->「Toggle hidden files」。在 **image_scoring** 資料夾中，建立含有以下內容的 .env 檔案。新增必要詳細資料，例如專案名稱和 Cloud Storage bucket (在 Cloud Shell 編輯器終端機中執行)。

```
#go to image_scoring folder
cd ~/imagescoring/multiagenthandson/image_scoring
```

```
cat <<EOF>> .env
GOOGLE_GENAI_USE_VERTEXAI=1
GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
GOOGLE_CLOUD_LOCATION=us-central1
GOOGLE_CLOUD_STORAGE_BUCKET=$(gcloud config get-value project)-imagescoring-bucket
GCS_BUCKET_NAME=$(gcloud config get-value project)-imagescoring-bucket
SCORE_THRESHOLD=40
IMAGEN_MODEL="imagen-3.0-generate-002"
GENAI_MODEL="gemini-2.5-flash"
EOF
```

5. 在 Cloud Shell 編輯器選單中，依序選取「File」>「Open Folder」。
6. 在彈出式方塊中，於使用者名稱後方新增下列資料夾資訊：`imagescoring/`。按一下「確定」。現在左側的檔案總管窗格中，應該會顯示完整的專案結構。
7. 在「Explorer」側邊窗格中，前往 `image_scoring` 資料夾。點選「agent.py」[agent.py](#)開啟檔案，查看代理程式結構。這個代理程式包含根代理程式，可連結至其他子代理程式。

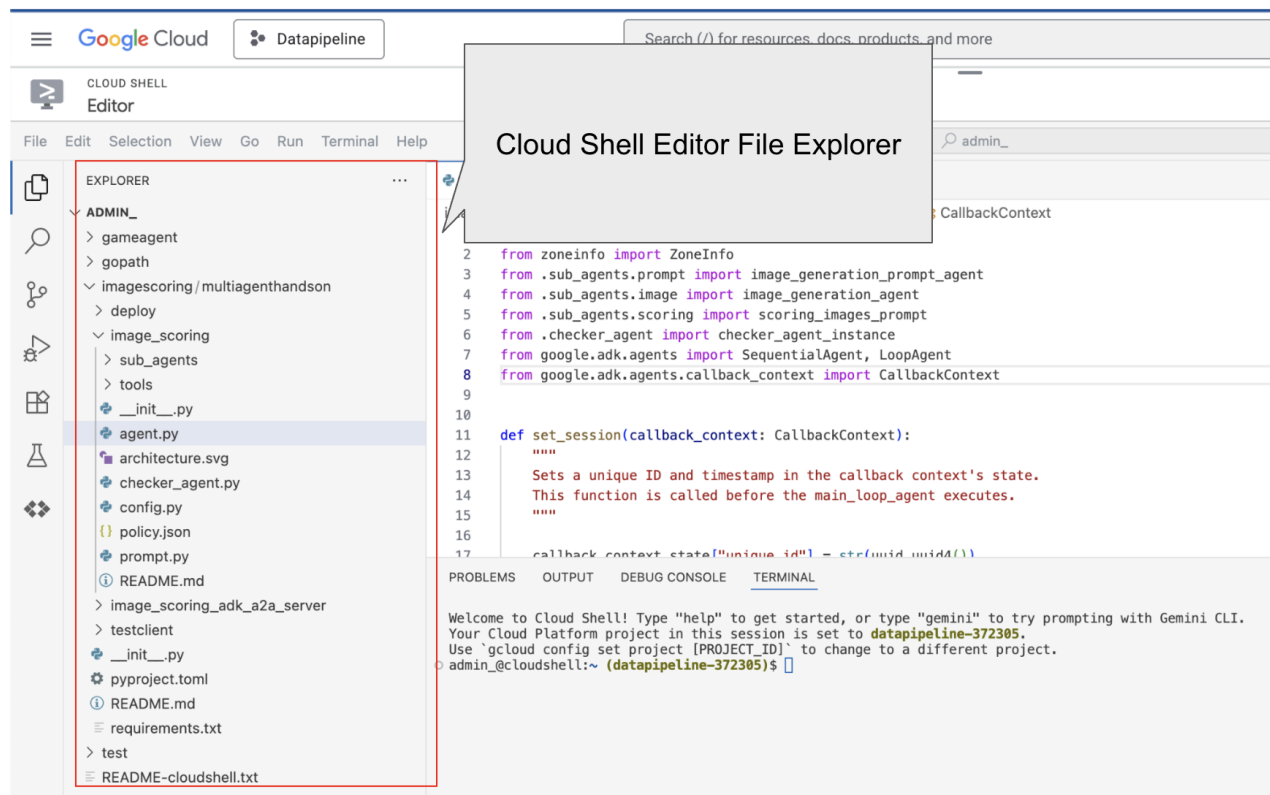


圖 4：從「Explorer」側邊窗格看到的資料夾結構。只要按一下檔案，即可查看檔案內容。

8. 返回終端機中的頂層目錄 **multiagenthandson**，然後執行下列指令，在本地執行代理程式 (在 Cloud Shell 編輯器終端機中執行)。

```
#go to the directory multiagenthandson
cd ~/imagescoring/multiagenthandson
# Run the following command to run agents locally
adk web
```

```
INFO:      Started server process [1875]
INFO:      Waiting for application startup.

+-----+
| ADK Web Server started                               |
|                                                      |
| For local testing, access at http://localhost:8000. |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

圖 5：啟動 ADK 應用程式

9. 在終端機上顯示的 [http://](http://localhost:8000) 網址上按住 **Ctrl** 鍵並點選 (MacOS 則為按住 **CMD** 鍵並點選)，即可開啟 ADK 的瀏覽器型 GUI 用戶端。畫面應如圖 5 所示
10. 選取左上側下拉式選單中的「image_scoring」(請參閱圖 5)。現在來生成幾張圖片吧！您也應該會在 Google Cloud Storage bucket 中找到圖片。請嘗試使用下列提示詞或自訂提示詞。

- 日落時分，寧靜的山景
- 貓騎自行車

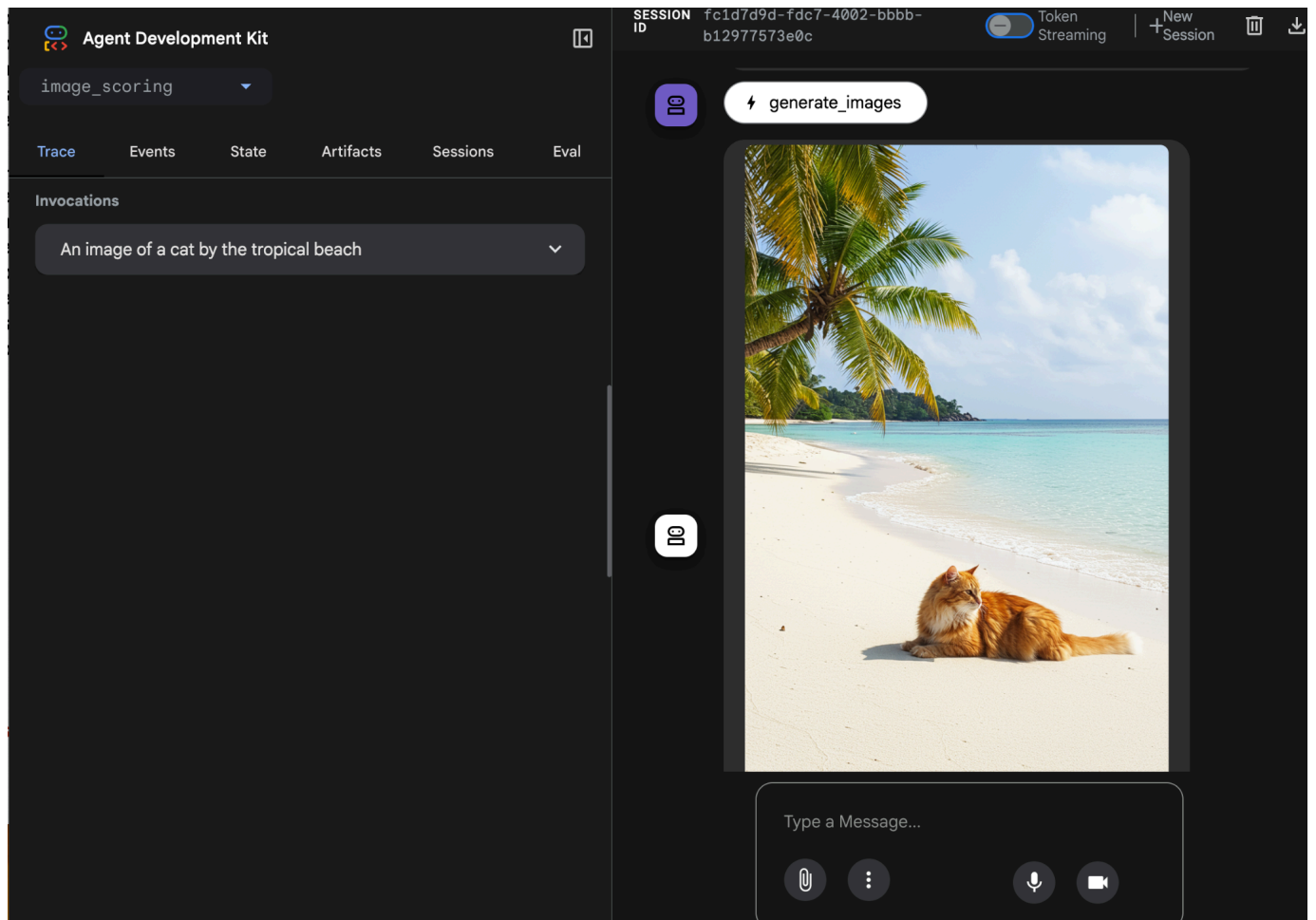


圖 6：輸出內容範例

9. 部署至 Agent Engine

現在，我們要將代理部署至 Agent Engine。[Agent Engine](#) 是一項全代管服務，可在 GCP 中部署代理程式。Agent Engine 與 [ADK \(Agent Development Kit\)](#) 相容，因此以 [ADK \(Agent Development Kit\)](#) 建構的代理可部署在 Agent Engine 中。

1. 在 Cloud Shell 編輯器終端機中執行下列步驟前，請先按下 **Ctrl+C** 鍵關閉 ADK 伺服器。
2. 使用 Poetry 建立 **requirements.txt** 檔案。Poetry 會使用 **pyproject.toml** 建立 **requirements.txt** 檔案。執行指令後，請檢查是否已建立 **requirements.txt** 檔案 (在 Cloud Shell 編輯器終端機中執行)。

```
# Go to the parent folder containing pyproject.toml file
cd ~/imagescoring/multiagenthandson

# install poetry-plugin-export
uv pip install poetry-plugin-export

#Create requirements.txt file
python3 -m poetry export -f requirements.txt --output requirements.txt --without-hashes
```

3. 建立套件。我們需要將應用程式打包成 **.whl** Python 套件。我們會使用 Poetry 執行這項操作。執行指令後，請確認是否已建立 dist 資料夾，且該資料夾包含 **.whl** 檔案 (在 Cloud Shell 編輯器終端機中執行)。

```
# Go to the parent folder containing pyproject.toml file
cd ~/imagescoring/multiagenthandson

#Create python package, to create whl file
python3 -m poetry build
```

4. 現在我們要準備將圖片評分代理部署至 Agent Engine 服務的指令碼。在 **deploy** 目錄中，於 Cloud Shell 編輯器側邊窗格找到 **deploy.py**，然後點選開啟。請按照下列步驟驗證內容


```

import vertexai
from image_scoring.agent import root_agent
import os
import glob # To easily find the wheel file
from dotenv import load_dotenv

# Load environment variables from image_scoring/.env
env_path = os.path.join(os.path.dirname(__file__), "..", "image_scoring", ".env")
load_dotenv(env_path)

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
LOCATION = os.getenv("GOOGLE_CLOUD_LOCATION", "us-central1")
STAGING_BUCKET = f"gs://{os.getenv('GOOGLE_CLOUD_STORAGE_BUCKET')}"

from vertexai import agent_engines

client=vertexai.Client(
    project=PROJECT_ID,
    location=LOCATION,
)
remote_app = client.agent_engines.create(
    agent=root_agent,
    config={
        "display_name": "image-scoring",
        "staging_bucket": STAGING_BUCKET,
        "requirements": open(os.path.join(os.getcwd(), "requirements.txt")).readlines() +
["./dist/image_scoring-0.1.0-py3-none-any.whl"],
        "extra_packages": [
            "./dist/image_scoring-0.1.0-py3-none-any.whl",
        ],
        "env_vars":{"GCS_BUCKET_NAME":os.getenv('GOOGLE_CLOUD_STORAGE_BUCKET')}
    }
)
print(f"DEBUG: AgentEngine attributes: {dir(remote_app)}")
try:
    print(remote_app.api_resource.name)
except AttributeError:
    print("Could not find resource_name, check DEBUG output above.")

```

5. 現在可以執行部署指令碼。首先，請前往頂層資料夾 **multiagenthandson** (在 Cloud Shell 編輯器終端機中執行)。

```

#go to multiagenthandson folder
cd ~/imagescoring/multiagenthandson

#run deploy script from the parent folder containing deploy.py
python3 -m deploy.deploy

```

部署完成後，您應該會看到類似下方的內容，

```
INFO:vertexai_genai.agentengines:Agent Engine created. To use it in another session:
INFO:vertexai_genai.agentengines:agent_engine=client.agent_engines.get(name='projects/85469421903/locations/us-central1/reasoningEngines/428031080600174592')
DEBUG: AgentEngine attributes: {'_abstractmethods': '', '_annotations': '', '_class': '', '_class_getitem': '', '_class_vars': '', '_copy': '', '_deepcopy': '', '_delattr': '', '_dict': '', '_dir': '', '_doc': '', '_eq': '', '_fields': '', '_fields_set': '', '_firstlineno': '', '_format': '', '_ge': '', '_get_pydan': '', '_get_core_schema': '', '_get_pydantic_json_schema': '', '_getattr': '', '_getattribute': '', '_getstate': '', '_gt': '', '_hash': '', '_init': '', '_init_subcl': '', '_iter': '', '_le': '', '_lt': '', '_module': '', '_ne': '', '_new': '', '_pretty': '', '_private_attributes': '', '_pydantic_complete': '', '_pydantic_c': '', '_computed_fields': '', '_pydantic_core_schema': '', '_pydantic_custom_init': '', '_pydantic_decorators': '', '_pydantic_extra': '', '_pydantic_fields': '', '_pydantic_fields_set': '', '_pydantic_generic_metadata': '', '_pydantic_init_subclass': '', '_pydantic_on_complete': '', '_pydantic_parent_namespace': '', '_pydantic_post_init': '', '_pydantic_private': '', '_pydantic_reort_model': '', '_pydantic_serializer': '', '_pydantic_setattr_handlers': '', '_pydantic_validator': '', '_reduce': '', '_reduce_ex': '', '_replace': '', '_repr': '', '_repr_args': '', '_repr_name': '', '_repr_recursion': '', '_repr_str': '', '_rich_h_repr': '', '_setattr': '', '_setstate': '', '_signature': '', '_sizeof': '', '_slots': '', '_static_attributes': '', '_str': '', '_subclasshook': '', '_weakref': '', '_abc_impl': '', '_calculate_keys': '', '_check_field_type_mismatches': '', '_copy_and_set_values': '', '_from_response': '', '_get_value': '', '_iter': '', '_setattr_handl': '', '_api_async_client': '', '_api_client': '', '_api_resource': '', '_construct': '', '_copy': '', '_delete': '', '_dict': '', '_from_orm': '', '_json': '', '_model_computed_fields': '', '_model_conf': '', '_model_construct': '', '_model_copy': '', '_model_dump': '', '_model_dump_json': '', '_model_extra': '', '_model_fields': '', '_model_fields_set': '', '_model_json_schema': '', '_model_parametrized_name': '', '_model_post_init': '', '_model_rebuild': '', '_model_validate': '', '_model_validate_json': '', '_model_validate_strings': '', '_operation_schemas': '', '_parse_e_file': '', '_parse_obj': '', '_parse_raw': '', '_schema': '', '_schema_json': '', '_to_json_dict': '', '_update_forward_refs': '', '_validate']
projects/85469421903/locations/us-central1/reasoningEngines/428031080600174592
```

圖 7：輸出範例

6. 現在來測試已部署的代理程式。如要測試遠端部署的代理程式引擎，請先從終端機的部署輸出內容複製代理程式位置。格式應類似：*projects/85469421903/locations/us-central1/reasoningEngines/7369674597261639680*。

在 Cloud Shell 編輯器的側邊窗格中，前往 **testclient** 資料夾，點選 **remote_test.py** 開啟檔案，然後編輯下列幾行：

```
REASONING_ENGINE_ID = "projects/xxx/locations/us-central1/reasoningEngines/xxx" # TODO:
Change this
```

7. 在 **multiagenthandsn** 根目錄中，於 Cloud Shell 編輯器終端機執行下列指令。輸出內容應與圖 8 相同。

```
#go to multiagenthandsn folder
cd ~/imagescoring/multiagenthandsn

#execute remote_test.py
python3 -m testclient.remote_test
```

```
s."",\n },\n "Safe Zones": {\n "score": 5,\n "reason": "The minimalist and clear background, combined with the focus on clock and notifi\n cation visibility, implies that safe zones are maintained to prevent cropping of important content."},\n "Composition Styles": {\n "sc\n ore": 5,\n "reason": "The image is generated with a focus on visual appeal and good composition, as suggested by the detailed and aesthetic posit\n ive prompt, ensuring adherence to composition styles."},\n "Color Scheme Definitions": {\n "score": 5,\n "reason": "The image utili\n zes a natural and harmonious color scheme with 'soft pastel tones' and 'warm natural sunlight,' ensuring good contrast and appealing color relativ\n e relationships."},\n "thought_signature": "Cq4BAEPx_17Ltn3nigIyuyH-DibCYHTMeQLND5toqUFviVFZDIqpNziTNwauSk57zsB526WFnpblMczIMEGJFST13yqOPawpf\n iealMSL22H06Bz3wLxwyo1U7GxN3E3WQwQ_dFXIm2bhYCYZ5SXP0FK54-6oD2iPKelbu-hURZvVs7MnrXzK0gqvsIXQDYKA7r25Hbu1jZJfLmChmR5Zpzaxu0T5rPIWq2yvHdMweYX"}},\n "role":\n 'model'},\n 'finish_reason': 'STOP',\n 'usage_metadata': {'cache_tokens_details': [{'modality': 'TEXT', 'token_count': 1832}], 'cached_content_token_cou\n nt': 1832, 'candidates_token_count': 559, 'candidates_tokens_details': [{'modality': 'TEXT', 'token_count': 559}], 'prompt_token_count': 2391, 'prompt\n _tokens_details': [{'modality': 'TEXT', 'token_count': 4687}], 'thoughts_token_count': 33, 'total_token_count': 2983, 'traffic_type': 'ON_DEMAND'},\n 'a\n vg_logprobs': -0.027370884508054458, 'invocation_id': 'e-1940d5f4-ab10-431d-836e-3aa6d9926506', 'author': 'scoring_images_prompt', 'actions': {'state_\n delta': {'scoring': 'I have evaluated the image against the provided scoring rules. Here is the detailed scoring breakdown and the total score:\n\n'''\n json{\n "total_score": 50,\n "scores": {\n "General Guidelines": {\n "score": 5,\n "reason": "The image of a cat does not violate any\n content restrictions, making it appropriate for general guidelines."},\n "Global Defaults": {\n "score": 5,\n "reason": "The image\n adheres to general good defaults for image content, composition style, and color scheme as intended by the detailed positive prompt."},\n "M\n edia Type Definitions": {\n "score": 5,\n "reason": "The image is a valid image media type and follows general content guidelines for images\n by being high quality and appropriate."},\n "Image Specifications and Guidelines": {\n "score": 5,\n "reason": "The image is descr\n ibed as ultra-detailed, photorealistic, and high-resolution, ensuring a premium, high-quality appearance appropriate for all audiences, fulfilling the\n image specifications."},\n "Text Specifications and Guidelines": {\n "score": 5,\n "reason": "The image does not contain any text,\n thereby adhering to the guidelines by not introducing any text-related issues and avoiding potential violations."},\n "Clock Visibility": {\n\n "score": 5,\n "reason": "The image prompt explicitly stated the inclusion of ample clear space for clock visibility, ensuring this criteri\n on is met."},\n "Notification Area": {\n "score": 5,\n "reason": "The image prompt explicitly stated the inclusion of ample clear s\n pace for notification visibility, with a minimalist and blurred background to avoid complex patterns."},\n "Safe Zones": {\n "score":\n 5,\n "reason": "The minimalist and clear background, combined with the focus on clock and notification visibility, implies that safe zones are\n maintained to prevent cropping of important content."},\n "Composition Styles": {\n "score": 5,\n "reason": "The image is generatec\n with a focus on visual appeal and good composition, as suggested by the detailed and aesthetic positive prompt, ensuring adherence to composition sty\n les."},\n "Color Scheme Definitions": {\n "score": 5,\n "reason": "The image utilizes a natural and harmonious color scheme with '\n soft pastel tones' and 'warm natural sunlight,' ensuring good contrast and appealing color relationships."},\n "artifact_delta":\n {},\n 'requested_auth_configs': {},\n 'requested_tool_confirmations': {},\n 'id': '5e8a4f41-ed80-4c41-9500-653d2fc09512',\n 'timestamp': '1764395366.790762'}\n {'model_version': 'gemini-2.5-flash', 'content': {'parts': [{'function_call': {'id': 'adk-3da3380c-9d6a-4183-9e31-11491a7aa6ac', 'args': {}, 'name': 'c\n heck_condition_and_escape_tool'},\n 'thought_signature': 'CoMFAEPx_17MRvHl_0s2AmT0wesQF614PqWGBARxeNam5PwEZPxJvXSuPwm1FKcLRKJtj4m2K8SaY1j2zxsRYN3WMT\n EMkEp9S8PltnyVux_YH2Z2bawbwEbCa5gZig0nGZ2I-h7P416v0dTNIY6ax1L3_DLeTfFcMMHFc-imlse-INc5Neat6w6Za1NoeaN0BJpdhXklUcTG9w3pGpRMKui-3UUsbCjQlBBRCz9TKHVE9p-2\n Q1R37Q6CdYXZiIqrTz1JdUsyJSE-zybk_N6HKqYPZwP_w2usjTqy6CKSqwDl15hK2o3CdJHqjVGdGFAVCw6NgvF1Nxs5gVkgdR_U9UfqNjUKXm10X4Kx0tuxYsRn2Y55yKAuHd1pdX0o1cVYuhCF\n VpsP7ng4yYn3B8TGIEEZKj6FdFp_X8KwQrd7YgCyWfXnpPLEoLCPBF6-ywI8KDCQyq6KBCfH499WycbNKGJHSPFaY721U1V101313DmophfEJLyTV_h2Tt1N98f20HvYV3_TPE8NEJD18FymkbnNz\n oI_qtdqhMbmEHkknLiQcx7bhauLm-8MsU-mFhkqsAGZU6nv5_-qaK4necLiWkAih5Yt1JH05PaIeYymQc1tVbM0MTj7BDldQxHfQkfyEzvtFLO_tE3evSLAVVTPgN-XEW56b9bqjBATrVjT7J\n V1Za1uQUX7cauVFG-K7jadVawGnf7HsrhG0aQpg8S9oKf7FZ2jTV1LFvG4SKRwJ-hT1_ADLzoxbq-wtUdLouRJ2N3Bzzazlse--aUj78ppg9UHNmGqUB1vJbsuqnehnLzjOvczFnM65ddtqhGuaAL3\n k8M9GcUzKbjJgm6lq5wyw=='}],\n 'role': 'model'},\n 'finish_reason': 'STOP',\n 'usage_metadata': {'candidates_token_count': 10, 'candidates_tokens_details': [\n {'modality': 'TEXT', 'token_count': 10}], 'prompt_token_count': 2416, 'prompt_tokens_details': [{'modality': 'TEXT', 'token_count': 2416}], 'thoughts\n _token_count': 160, 'total_token_count': 2586, 'traffic_type': 'ON_DEMAND'},\n 'avg_logprobs': -4.455498886108399, 'invocation_id': 'e-1940d5f4-ab10-431c\n -836e-3aa6d9926506', 'author': 'checker_agent', 'actions': {'state_delta': {}, 'artifact_delta': {}, 'requested_auth_configs': {}, 'requested_tool_cor\n firmations': {},\n 'long_running_tool_ids': [],\n 'id': '340d95b9-3319-40a2-a2f6-1f53d159f710',\n 'timestamp': '1764395370.374722'}\n version <\n 4.0'\n\n',\n 'pyasn1-modules==0.4.2 ; python_version >= "3.10" and python_version < "4.0"',\n 'pyasn1==0.6.1 ; python_version >= "3.10" and python_versio\n n < "4.0"',\n 'pyparsing==2.23 ; python_version >= "3.10" and python_version < "4.0"',\n 'platform_python_implementation\n != "PyPy" and python_version >= "3.10" and python_version < "4.0"',\n 'platform_python_implementation\n != "PyPy" and python_version >= "3.10" and python_version < "4.0"',\n 'pydantic-core==2.41.5 ; python_version >= "3.10" and python_version < "4.0"',\n 'pydantic-settings==2.12.0 ; python_version >= "3.1\n 0" and python_version < "4.0"',\n 'pydantic==2.12.5 ; python_version >= "3.10" and python_version < "4.0"',\n 'pyjwt==2.10.1 ; python_version >= "3.10\n " and python_version < "4.0"',\n 'pyyaml==3.2.5 ; python_version >= "3.10" and python_version < "4.0"',\n 'python-dateutil==2.9.0.post0 ; python_v\n rsion >= "3.10" and python_version < "4.0"',\n 'python-dotenv==1.2.1 ; python_version >= "3.10" and python_version < "4.0"',\n 'python-multipart==0.0.\n 20 ; python_version >= "3.10" and python_version < "4.0"',\n 'pytz==2025.2 ; python_version >= "3.10" and python_version < "4.0"',\n 'pywin32==311 ; p\n ython_version >= "3.10" and python_version < "4.0"',\n 'sys_platform < "win32"',\n 'pyyaml==6.0.3 ; python_version >= "3.10" and python_version < "4.0\n "',\n 'referencing==0.37.0 ; python_version >= "3.10" and python_version < "4.0"',\n 'regex==2025.11.3 ; python_version >= "3.10" and python_version < "4.0\n "',\n 'requests==2.32.5 ; python_version >= "3.10" and python_version < "4.0"',\n 'rouge-score==0.1.2 ; python_version >= "3.10" and python_versi\n on < "4.0"',\n 'rpds-py==0.29.0 ; python_version >= "3.10" and python_version < "4.0"',\n 'rsa==4.9.1 ; python_version >= "3.10" and python_version < "4.0\n "',\n 'ruamel-yaml-clib==0.2.15 ; python_version >= "3.10" and python_version >= "3.13" and platform_python_implementation ==
```

圖 8：輸出範例

10. 建立 A2A 代理程式

在本步驟中，我們將根據上一個步驟建立的代理，建立簡單的 A2A 代理。現有的 [ADK \(Agent Development Kit\)](#) 代理可透過 [A2A](#) 通訊協定發布。這些是您將在本步驟中學到的重點。

- 瞭解 [A2A](#) 通訊協定的基本概念。
- 瞭解 ADK 和 [A2A](#) 通訊協定如何搭配運作。
- 瞭解如何與 [A2A](#) 通訊協定互動。

在本實作練習中，我們將使用 `image_scoring_adk_a2a_server` 資料夾中的程式碼。開始工作前，請將目錄變更為這個資料夾 (在 Cloud Shell 編輯器終端機中執行)。

```
#change directory to image_scoring_adk_a2a_server
cd ~/imagescoring/multiagenthandson/image_scoring_adk_a2a_server

#copy the env file
cp ~/imagescoring/multiagenthandson/image_scoring/.env remote_a2a/image_scoring
```

1. 建立 A2A 代理資訊卡

[A2A](#) 通訊協定需要[代理程式資訊卡](#)，其中包含代理程式的所有資訊，例如代理程式功能、代理程式使用指南等。部署 [A2A](#) 代理程式後，即可使用「`.well-known/agent-card.json`」連結查看代理程式資訊卡。客戶可以參考這項資訊，將要求傳送給服務專員。

前往 `remote_a2a/image_scoring` 目錄，然後在 **Cloud Shell 編輯器**側邊窗格中找到 `agents.json`。按一下檔案開啟，並確認內容與下列內容相符：

```
{
  "name": "image_scoring",
  "description": "Agent that generates images based on user prompts and scores their adherence to the prompt.",
  "url": "http://localhost:8001/a2a/image_scoring",
  "version": "1.0.0",
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["image/png", "text/plain"],
  "capabilities": {
    "streaming": true,
    "functions": true
  },
  "skills": [
    {
      "id": "generate_and_score_image",
      "name": "Generate and Score Image",
      "description": "Generates an image from a given text prompt and then evaluates how well the generated image adheres to the original prompt, providing a score.",
      "tags": ["image generation", "image scoring", "evaluation", "AI art"],
      "examples": [
        "Generate an image of a futuristic city at sunset",
        "Create an image of a cat playing a piano",
        "Show me an image of a serene forest with a hidden waterfall"
      ]
    }
  ]
}
```

2. 建立 A2A 代理 在 `image_scoring_adk_a2a_server` 根目錄中，確認 `a2a_agent.py` 檔案是否存在。如要開啟，請在 Cloud Shell 編輯器的側邊窗格中點選檔案名稱。這個檔案是 A2A 代理的進入點，應包含下列內容：

```
#change directory to image_scoring_adk_a2a_server
cd ~/imagescoring/multiagenthandson/image_scoring_adk_a2a_server
```

```
from google.adk.agents.remote_a2a_agent import RemoteA2aAgent

root_agent = RemoteA2aAgent(
    name="image_scoring",
    description="Agent to give interesting facts.",
    agent_card="http://localhost:8001/a2a/image_scoring/.well-known/agent.json",

    # Optional configurations
    timeout=300.0,          # HTTP timeout (seconds)
    httpx_client=None,      # Custom HTTP client
)
```

3. 執行 A2A 代理

現在可以執行代理程式了！如要執行代理程式，請在頂層資料夾 `image_scoring_adk_a2a_server` 中執行下列指令 (在 Cloud Shell 編輯器終端機中執行)。

```
#following command runs the ADK agent as a2a agent
adk api_server --a2a --port 8001 remote_a2a
```

4. 測試 A2A 代理

代理程式執行後，我們就可以測試代理程式。首先，請查看代理程式資訊卡。使用「Terminal」(終端機) > 「New Terminal」(新增終端機) 開啟新的終端機，然後執行下列指令 (在開啟的 Cloud Shell 編輯器終端機中執行)。

```
#Execute the following
curl http://localhost:8001/a2a/image_scoring/.well-known/agent.json
```

執行上述指令後，應該會顯示 [A2A](#) 代理程式的代理程式資訊卡，這主要是我們在上一個步驟中建立的 **agent.json** 內容。

現在，讓我們向代理傳送要求。我們可以使用 curl 將要求傳送至代理程式 (在剛開啟的 Cloud Shell 編輯器終端機中執行)，

```
curl -X POST http://localhost:8001/a2a/image_scoring -H 'Content-Type: application/json' \
-d '{
  "id": "uuid-123",
  "params": {
    "message": {
      "messageId": "msg-456",
      "parts": [{"text": "Create an image of a cat"}],
      "role": "user"
    }
  }
}'
```

在上述要求中，您可以變更「**Create an image of a cat**」這一行，藉此變更提示。執行指令後，您可以在指定的 [Google Cloud Storage](#) 中查看輸出圖片。

11. 清除所用資源

現在來清理剛建立的內容。

- 1. 刪除我們剛建立的 [Vertex AI Agent Engine](#) 伺服器。在 Google Cloud 控制台的搜尋列中輸入「Vertex AI」，前往 Vertex AI。按一下左側的「代理程式引擎」。按一下「刪除」即可刪除代理程式。

Filter

Name	Created	Framework	Description
二	Aug 1, 2025, 8:54:25 AM	google-adk	
二	Jul 28, 2025, 12:09:20 PM	google-adk	Edit
二	Jul 24, 2025, 6:55:40 PM	google-adk	Delete
一	Jul 24, 2025, 4:42:25 PM	—	

圖 9：您可以從 Google Cloud 控制台刪除 Vertex AI Agent Engine 執行個體

- 2. 刪除 Cloud Shell 中的檔案

```
#Execute the following to delete the files
cd ~
rm -R ~/imagescoring
```

- 3. 刪除 bucket。前往 GCP 控制台的「Cloud Storage」，選取並刪除 bucket。

	Name ↑	Created	Location type	Location	Default storage	
☒	85469421903_asia_northeast1_impo...	Dec 18, 2023, 3:36:50 PM	Region	asia-northeast1	Standard	⋮
☐	85469421903_us_central1_import_cu...	Jun 16, 2024, 3:33:04 PM	Region	Pin bucket		
☐	85469421903_us_east1_import_cont...	Aug 11, 2023, 6:45:19 PM	Region	Edit access		
☐	85469421903_us_east1_import_cont...	Jan 30, 2024, 9:55:22 AM	Region	Edit tags		
☐	85469421903_us_east1_import_cust...	Mar 12, 2024, 10:48:46 AM	Region	Edit labels		
☐	85469421903_us_east1_import_docu...	Sep 16, 2024, 10:50:41 PM	Region	Edit website configuration		
☐	85469421903_us_import_content_wit...	Jul 2, 2024, 9:04:44 AM	Multi-region	Edit default storage class		
☐	artifacts.datapipeline-372305.appspo...	Apr 22, 2023, 1:16:44 PM	Multi-region	Delete bucket		
☐	cloud-ai-platform-51c3b92d-16f7-44f...	Jan 21, 2023, 9:19:00 PM	Region	Transfer data in		

圖 10：刪除 bucket

12. 結論

恭喜！您已成功將多代理 [ADK \(Agent Development Kit\)](#) 應用程式部署至 [Vertex AI Agent Engine](#)。這項重大成就涵蓋現代雲端原生應用程式的核心生命週期，為您部署複雜的代理程式系統奠定堅實基礎。

重點回顧

在本實驗室中，您學會如何：

- 使用 [ADK \(Agent Development Kit\)](#) 建立多代理應用程式
- 將應用程式部署至 [Vertex AI Agent Engine](#)
- 建立可使用 [A2A](#) 通訊協定通訊的代理。

實用資源

- [官方 ADK 說明文件](#)
- [Vertex AI Agent Engine](#)
- [A2A 通訊協定](#)
- [Cloud Run](#)

從原型設計到投入正式環境

這個實驗室屬於「[Google Cloud 學習路徑：打造可用於正式環境的 AI](#)」。

- [探索完整課程](#)，從設計原型開始，一步步把專案投入正式環境。
 - 使用 [#ProductionReadyAI](#) 主題標記分享進度。
-